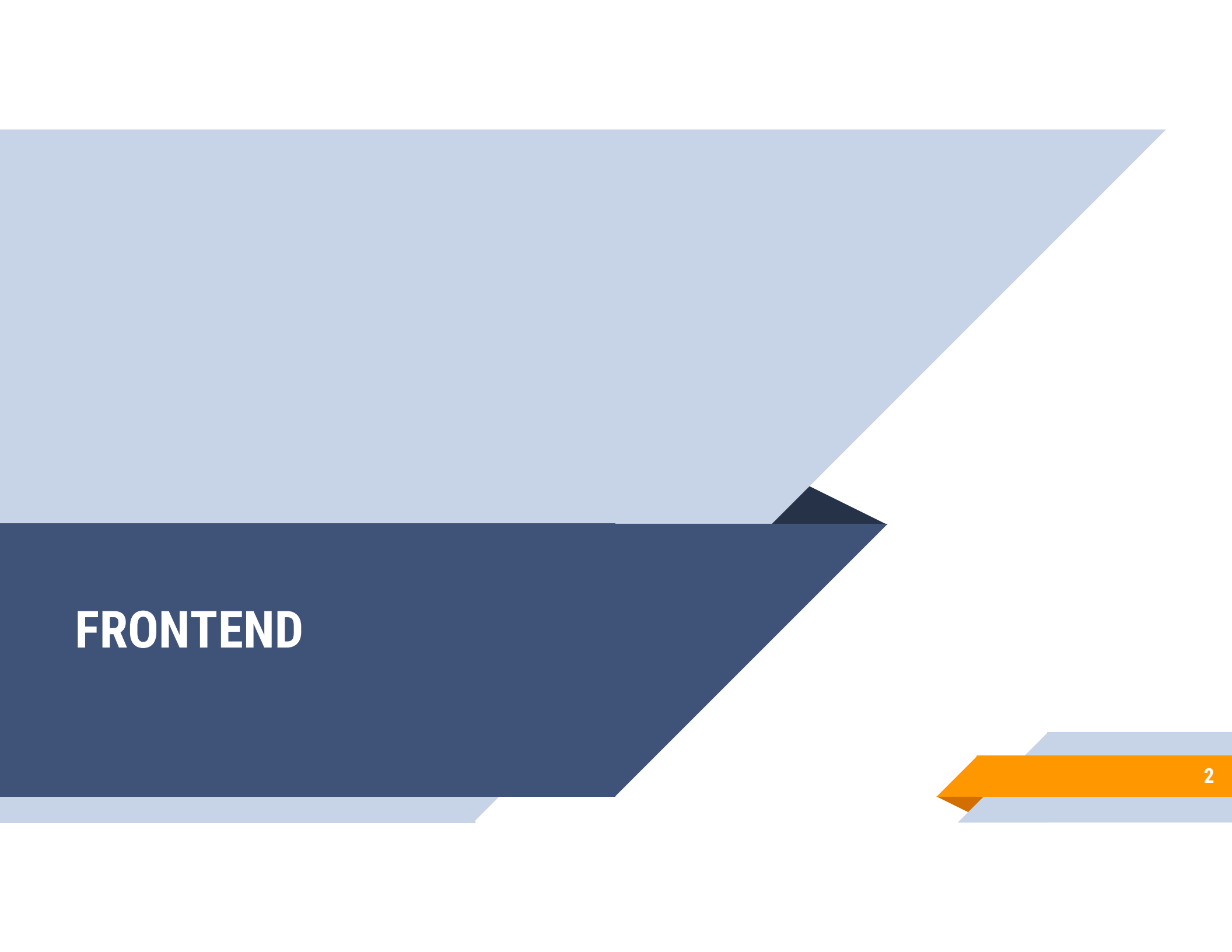


LẬP TRÌNH FRONTEND

The background features a large, dark blue chevron shape pointing to the right, which contains the text. Above and below this shape are lighter blue horizontal bands. At the bottom right, there is a bright orange horizontal bar that also has a slight 3D effect.

The image features a minimalist, abstract design. A large, light blue trapezoidal shape occupies the upper left portion of the frame. Below it, a dark blue trapezoidal shape extends horizontally across the middle. The word "FRONTEND" is written in white, bold, sans-serif capital letters on the left side of this dark blue shape. In the bottom right corner, there is a small orange trapezoidal shape. The background is a very light blue, and the overall composition is clean and modern.

FRONTEND



Frontend



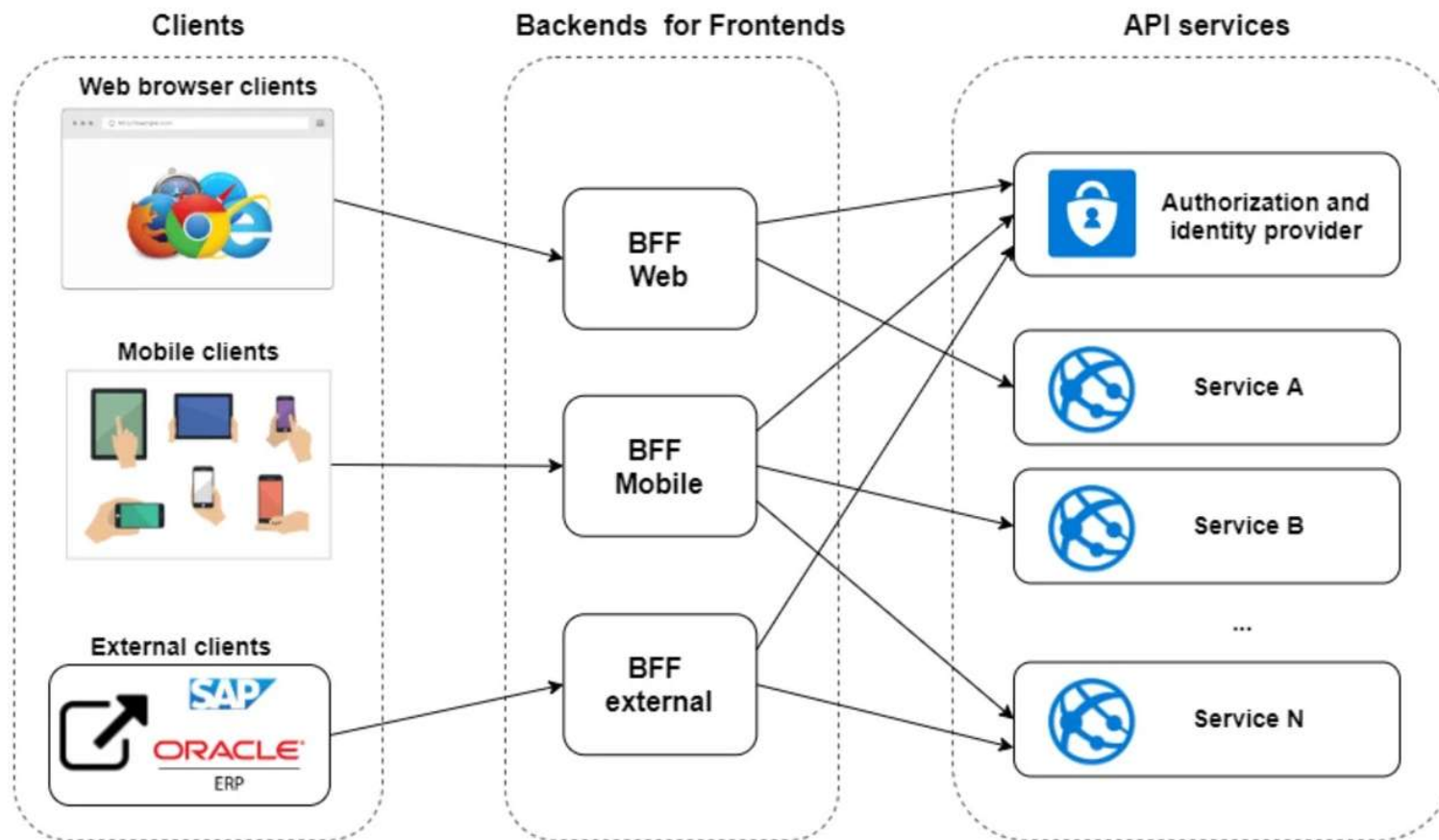
Frontend

- Tương tác với người dùng (sự kiện chuột, bàn phím, cảm ứng, ...)
- Xử lý dữ liệu nhập (validation, sanitization, ...)
- Định dạng dữ liệu xuất (aggregation, filter, sort, format...)

Frontend == Client ?

- Frontend/backend $\neq$ client/server
- Ứng dụng nhỏ/đơn giản: cả front end và backend có thể cùng chạy trên client
- Ứng dụng lớn/phức tạp: có thể cần server riêng cho frontend (code frontend chạy trên cả client và server) gọi là Backend For Frontend

Backend For Frontend



REACT (REACTJS)

React (ReactJS)

- Phát triển bởi Facebook (2011)
- Open source (2013)
- Là một thư viện JS dùng để tạo ra các component có thể tái sử dụng
- Sử dụng virtual DOM để quản lý trạng thái các component, chỉ render lại những component nào có sự thay đổi

Component

Video.js

```
function Video({ video }) {  
  return (  
    <div>  
      <Thumbnail video={video} />  
      <a href={video.url}>  
        <h3>{video.title}</h3>  
        <p>{video.description}</p>  
      </a>  
      <LikeButton video={video} />  
    </div>  
  );  
}
```



My video
Video description



Reusable

VideoList.js

```
function VideoList({ videos, emptyHeading }) {  
  const count = videos.length;  
  let heading = emptyHeading;  
  if (count > 0) {  
    const noun = count > 1 ? 'Videos' : 'Video';  
    heading = count + ' ' + noun;  
  }  
  return (  
    <section>  
      <h2>{heading}</h2>  
      {videos.map(video =>  
        <Video key={video.id} video={video} />  
      )}  
    </section>  
  );  
}
```

3 Videos



First video

Video description



Second video

Video description

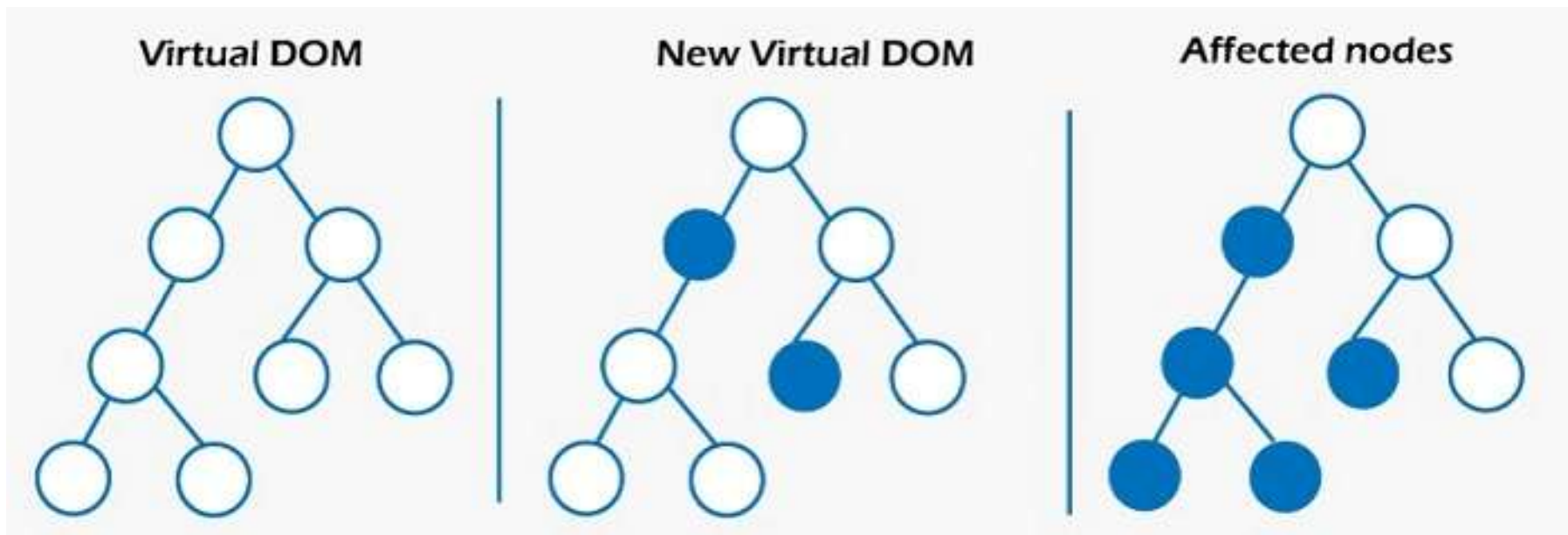


Third video

Video description



Virtual DOM



Development



Create React App



Vite

Tạo project

```
> npx create-react-app my-app  
> cd my-app
```

```
> npm create vite@latest my-app --  
--template react  
> cd my-project  
> npm install
```

Chạy trên trình duyệt

```
> npm start
```

```
> npm run dev
```

Build

```
> npm run build
```

```
> npm run build
```

Development

Các file người dùng có thể truy xuất trực tiếp, không cần build (media, static HTML, css, favico, robot.txt, ...)

Component chính của ứng dụng, thường dùng để chứa các component khác

```
> node_modules
> public
▼ src
  # App.css
  App.jsx
  JS App.test.js
  # index.css
  JS index.js
  logo.svg
  JS reportWebVitals.js
  JS setupTests.js
  .gitignore
  {} package-lock.json
  {} package.json
  README.md
```

Các file code cần build trước khi run/deploy

File JS chính để khởi động ứng dụng

Khai báo các package cần dùng

JSX

JSX

- JavaScript XML
- Là cú pháp để viết code HTML trong JS
- Tương tự HTML nhưng yêu cầu phải đóng các thẻ rỗng (<input />, ,
, ...)

JSX

```
function MyButton() {  
  return (  
    <button>I'm a button</button>  
  );  
}  
  
function AboutPage() {  
  return (  
    <>  
    <h1>About</h1>  
    <p>Hello there.<br />How do you do?</p>  
    </>  
  );  
}
```


JSX

- Đặt trong cặp dấu ()
- Chỉ có thể trả về 1 thẻ HTML
- Khi cần trả về nhiều thẻ (cùng cấp) thì phải lồng trong 1 thẻ HTML khác hoặc thẻ `<Fragment></Fragment>` (viết tắt `<></>`)
- `<Fragment></Fragment>` sẽ **không** xuất hiện trong HTML

JSX

- Tên các thuộc tính của thẻ HTML phải tránh các từ khóa (for => htmlFor, class => className, ...) và chuyển thành dạng camelCase
- Riêng các thuộc tính aria-* và data-* vẫn giữ nguyên

JSX

- Dùng cặp dấu { } để xuất giá trị biểu thức JS
- Đối với thuộc tính style: có thể xuất thẳng đối tượng JS thay vì chuyển thành dạng chuỗi CSS (lưu ý tên thuộc tính CSS dạng camelCase)

JSX

```
<img  
  className="avatar"  
  src={user.imageUrl}  
  alt={'Photo of ' + user.name}  
  style={{  
    width: user.imageSize,  
    height: user.imageSize  
  }}  
>
```

JSX

`<textarea>Hello world</textarea>`



`<textarea value="Hello world"></textarea>`

`<select>
 <option value="1">Option 1</option>
 <option value="2" selected>Option 2</option>
 <option value="3">Option 3</option>
</select>`



`<select value="2">
 <option value="1">Option 1</option>
 <option value="2">Option 2</option>
 <option value="3">Option 3</option>
</select>`

`<input type="checkbox" checked />
<input type="radio" name="radio" checked />`

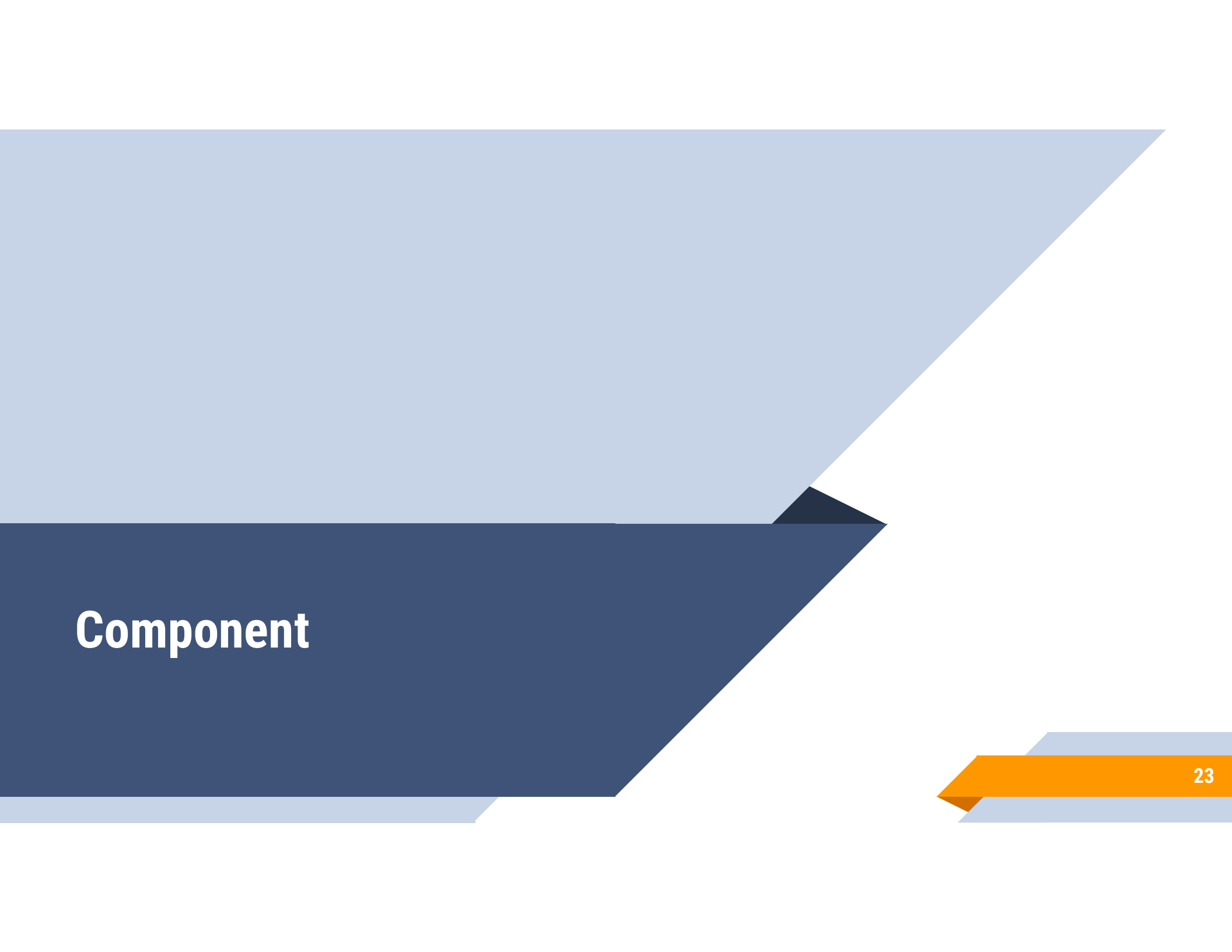


`<input type="checkbox" checked={true} />
<input type="radio" name="radio" checked={true} />`

Conditional rendering

```
<div>
  {isLoggedIn ? (
    <AdminPanel />
  ) : (
    <LoginForm />
  )}
</div>
```

```
<div>
  {isLoggedIn && <AdminPanel />}
</div>
```

The background features a large, light blue trapezoidal shape on the left side. Below it is a dark blue trapezoidal shape that tapers to the right. In the bottom right corner, there is a small orange trapezoidal shape. The word "Component" is written in white on the dark blue shape.

Component

Component

- Có thể dùng cú pháp class (cũ) hoặc function (mới)
- Tên phải bắt đầu bằng ký tự hoa
- Không được định nghĩa lồng nhau

Component



```
function A(){  
  return (<div>A</div>);  
}  
  
function B(){  
  return (  
    <>  
    <div>B</div>  
    <A />  
    </>  
  );  
}
```



```
function B() {  
  function A() {  
    return <div>A</div>;  
  }  
  return (  
    <>  
    <div>B</div>  
    <A />  
    </>  
  );  
}
```